Securing Agent-Generated Code Through Development History

Prateek Rajput*, Shahzeb Khan†, Jacques Klein*, and Tegawendé F. Bissyandé*
*University of Luxembourg, Luxembourg
{prateek.rajput, jacques.klein, tegawende.bissyande}@uni.lu
†Nostrum Biodiscovery S.L., Barcelona, Spain
shahzeb.khan@nostrumbiodiscovery.com

***Abstract*—Agentic software development makes it possible to capture the process of code creation, moving beyond static comments and commit messages to preserve edits, tool interactions, and recorded ReAct-style reasoning behind implementation decisions. We present TRACER (Trace-based Runtime Analysis for Continuous Evolution and Release), a provenance-aware workflow that links task descriptions and recorded agent rationale to the evolution of code and its constituent symbols. We ask whether combining these reasoning traces with code evolution helps investigators understand implementation assumptions and better diagnose and fix security flaws. TRACER preserves diagnostic evidence, repair rationale, and associated executable boundary checks alongside each repair, providing persistent context for future updates, refactors, and feature additions. Across 3,096 repair runs on the SusVibes benchmark, TRACER's change-informed planning covers reference-vulnerable symbols in 94.6% of attempts, compared with 13.4% for stock SAST locations. The rationale-enabled configuration achieves the highest verified security pass rates among the evaluated configurations with both investigators, while a stage-level analysis exposes where remaining candidates are lost during diagnosis and validation. Extending TRACER to real-world structural-biology repositories, we verify five component repairs and evaluate subsequent feature evolution under silent-fix conditions, where pairing persistent repair memory with a boundary gate and one feedback retry improves joint feature-and-security-check success from 3/5 to 5/5.**

***Index Terms*—Agentic Software Engineering, Vulnerability Repair, Provenance and Reasoning Traces, Silent Security Fixes, Continuous Software Evolution, Static and Dynamic Analysis.**

I. INTRODUCTION

Autonomous coding agents can navigate repositories, edit source files, and execute test suites through interfaces such as SWE-agent [1], while agent-based repair systems extend this tool-driven workflow to debugging [2]. For engineering teams adopting these systems, security remains a critical review obligation. Empirical studies of generated code and coding-assistant use document security weaknesses [3]–[5]. A patch can satisfy functional tests while leaving an unsafe input boundary, incomplete validation, or a violated API contract. SusVibes makes this gap explicit by evaluating functionality and security separately on repository-level tasks [6], [7]. Teams accepting working, agent-generated changes therefore need to uncover and repair omitted security conditions without breaking the requested functionality.

Existing software engineering defenses provide several starting points. Static application security testing (SAST) tools such as CodeQL [8] and Semgrep [9] supply reproducible rules, source locations, and flow evidence. Industrial deployments and developer studies emphasize integrating actionable diagnostics into code review [10]–[12]. LLM-assisted taint and dataflow analysis [13]–[15], analyzer-guided repair [16], and program-analysis-guided proof-of-concept generation [17] further connect model judgments to source and execution evidence. These techniques help examine the implementation under review, but current-source findings alone do not explain why a coding agent omitted a guard or what it assumed about upstream inputs. When scanner locations also miss the relevant changed symbols, an investigator must first establish where to look and then reconstruct the missing security condition.

Agentic development offers the recorded process of code creation as an additional source of evidence. A trajectory can preserve intermediate edits, tool interactions, and interleaved ReAct-style explanations [18], extending the evidence available from a final diff and commit message. Fine-grained AST differencing makes structural changes explicit [19], while recorded rationale supplies statements about input trust, callers, and library behavior. Together, these channels can expose a removed guard, an asymmetry between sibling functions, or a mismatch between an agent's stated assumption and its implementation. We treat explanations as hypotheses to check against current source, since generated rationale need not faithfully reveal internal reasoning [20], [21]. The opportunity is to connect the history of a change to the present security contract that a repair must satisfy.

Repair is also a maintenance problem. Work on silent vulnerability fixes shows that security-relevant changes can lack an explicit security designation [22], and subsequent edits can undo an earlier repair, as explored in security-regression studies [23]. A later refactor or feature addition may preserve the requested behavior while losing a condition whose purpose is no longer apparent. The same provenance collected during investigation can carry that purpose forward. The diagnostic evidence, enforced invariant, and independently owned executable boundary check can remain linked to the repaired symbol. A maintenance workflow can then test subsequent changes against the saved contract and return concrete feedback when they violate it.

We present TRACER (*Trace-based Runtime Analysis for Continuous Evolution and Release*), a provenance-aware secu-

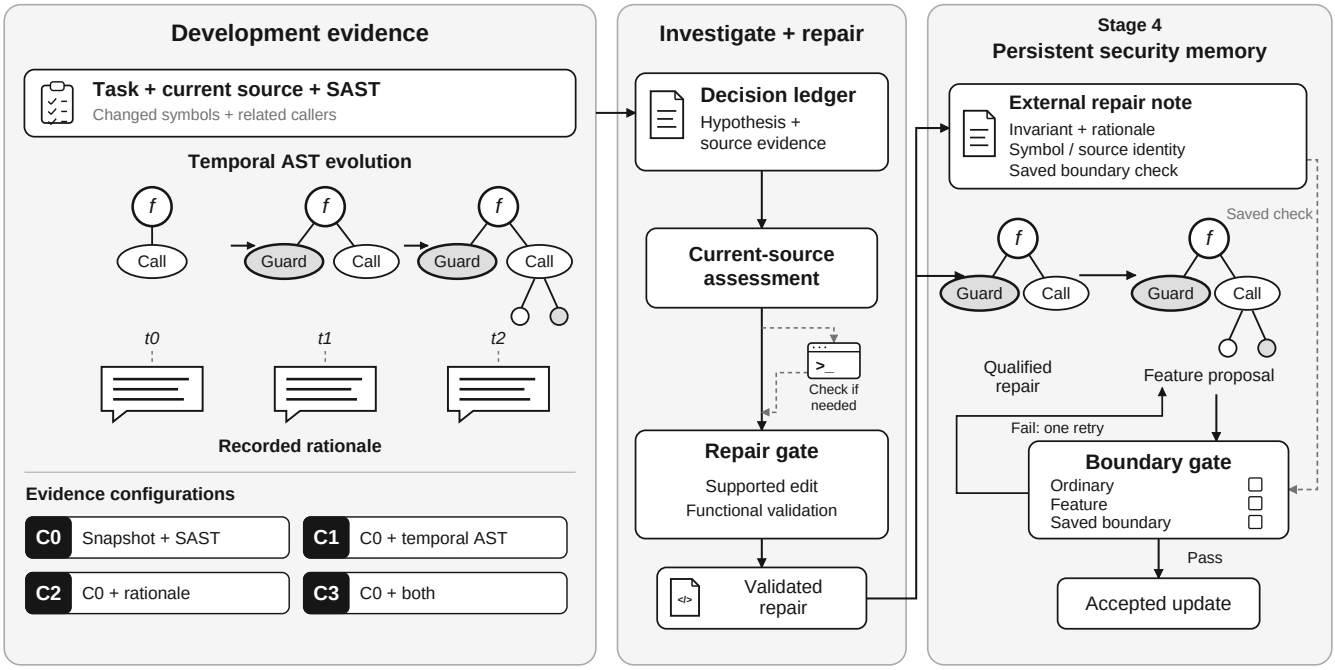


Fig. 1. TRACER links symbol-level development evidence to guarded security repair, then preserves repair invariants and executable checks for subsequent feature evolution. C0–C3 vary the evidence available during investigation.

rity investigation, repair, and maintenance workflow packaged as a lightweight, portable skill for integration with coding agents and continuous-integration pipelines. Given a bounded Git range or a compatible agent trajectory, TRACER performs change-informed symbol planning, keeps typed AST evolution separate from recorded rationale, grounds repair decisions in current-source assessment and conditional runtime checks, and emits persistent repair notes that can be coupled with executable boundary gates. Across 3,096 SusVibes repair runs spanning seven coding generators and two security investigators, its planner includes reference-vulnerable symbols in 94.6% of attempts, compared with 13.4% primary SAST symbol overlap. The rationale-enabled configuration achieves the highest verified security pass counts with both investigators. Combining both history channels does not improve on it. A complementary study of structural-biology software [24]–[29] verifies five component repairs and tests their subsequent feature evolution. Persistent memory with a one-retry boundary gate achieves joint feature-and-boundary success in 5/5 cases, compared with 3/5 for unguarded continuation.

The paper makes three contributions.

- **A provenance-aware architecture and portable skill.** TRACER connects change-informed planning, separate AST-evolution and rationale channels, guarded static/dynamic investigation, and persistent repair memory in a workflow that can accompany an agent-generated change through later maintenance.
- **A multi-investigator evaluation.** The SusVibes study compares four context bundles across seven coding generators and two investigators, measuring review coverage, verified security outcomes, and functional preservation. The structural-biology study evaluates the maintenance stage under silent-fix feature evolution.
- **A stage-level funnel and failure taxonomy.** A seven-version engineering progression and a full-cohort investigation funnel identify losses during diagnosis, validation scheduling, patch retention, and semantic repair, providing concrete design lessons for agentic review systems.

II. BACKGROUND AND MOTIVATION

A. Industry Relevance and Code Archaeology

When an agent-generated pull request works, an industrial reviewer must still determine which implementation assumptions deserve scrutiny and which security conditions must survive later maintenance. CodeQL and Semgrep can report locations and flows in the final snapshot [8], [9], but the reviewer must connect those findings to the submitted change, affected callers, and the intended contract. We call this reconstruction *code archaeology*. It recovers the context behind the current implementation from the evidence left available. Industrial static-analysis studies emphasize actionable diagnostics, developer trust, and integration into existing review workflows [10]–[12]. Preserving the development trace can supply context for that review, especially when the final diff omits an intermediate decision or correction.

For scientific-software teams such as Nostrum Biodiscovery, checkpoint loading, structure parsing, archive extraction, and output-path handling are concrete interfaces where compat-

ibility requests can invalidate earlier assumptions. A useful workflow must investigate these boundaries while preserving the requested scientific functionality, then retain the repair's purpose for the next change. Connecting review evidence to reusable checks fits secure-development practices for code review, testing, and retaining vulnerability knowledge [30]. Our domain study examines these component-level obligations. Production effectiveness remains to be evaluated.

B. Tracking Structural Evolution Within a Bounded Context

Making history available is not equivalent to making it usable. Source lines shift, definitions are added or renamed, and refactoring changes how code is divided among functions and classes. Intermediate fragments may not parse. A temporal representation therefore cannot assume that node names or line positions remain stable across versions. A line-based `git diff` records textual changes, but does not itself establish which function-level contract changed. Retaining every full AST and cross-version relation also expands the context with repeated or irrelevant structure. Fine-grained AST matching through GumTree [19], [31], together with function-change representations such as CoLeFunDa's [22], motivates a focused representation that maps observed changes to symbols, summarizes their structural edits, and preserves explicit gaps where continuity cannot be established.

C. Task Intent and Recorded Reasoning

Static analysis and AST differencing answer complementary questions about reported flows and structural changes, but neither alone supplies the requested behavior or every domain-specific assumption. Code can execute successfully while mishandling an input representation, trusting a caller without justification, or implementing only part of the task's contract. ReAct-style trajectories interleave explanations with actions [18], allowing the task description, stated assumptions, and observed edits to be examined together. These recorded explanations are auditable claims, not guaranteed accounts of internal chain-of-thought [20], [21]. A statement that an input is already validated, for example, creates a concrete obligation to inspect the relevant validator and its callers.

Our motivating cases come from repository tasks in SusVibes [6], [7]. In Python-RSA,[1] comparing `decrypt` and `verify` exposes an asymmetric encoded-input-length guard. LangChain[2] similarly illustrates an incomplete origin condition that checks network location but omits scheme equality. Wagtail CMS[3] shows the danger of repairing an intermediate label without inspecting downstream rendering. An early candidate introduces double-escaping. In Django,[4] recorded rationale favors character-reference conversion in `strip_tags`, while its tested contract requires preserving relevant text representations. A partial handler repair still fails. These cases motivate both discovering missing invariants and checking a proposed repair at the correct boundary.

[1]https://github.com/sybrenstuvel/python-rsa
[2]https://github.com/langchain-ai/langchain
[3]https://github.com/wagtail/wagtail
[4]https://github.com/django/django

III. TRACER Architecture and Portable Skill

A. Symbol-Level Tracking and Snapshot SAST (C0)

The evaluated Python pipeline builds a symbol-level index using the standard-library `ast.parse` parser [32] and a custom visitor in `planner.py`. For each parsed snapshot, the index records functions, asynchronous functions, methods, and classes by repository path, qualified name, and definition span. The history adapter also records structural hashes. Rather than using diff lines as the review unit, the planner maps submitted patch hunks and deletion anchors to their enclosing current symbols, excludes test and documentation paths, and records module-level or unavailable mappings when needed. It links guards, candidate in-file callers, and related changed symbols, while attributing Semgrep and CodeQL locations and flow evidence to their originating scanners [8], [9]. Local call-name matches are syntactic review hints, not resolved dataflow. This channel defines C0 as a change-informed snapshot review that scopes investigation beyond scanner locations, but still requires archaeology to explain how and why the current symbol was written.

B. Temporal AST Evolution via GumTree (C1)

C1 adds the observed structural evolution of selected symbols. TRACER parses available before/after source pairs into typed Python ASTs and computes edit scripts with GumTree 3.0.0 [19], [31]. It filters operations to the target symbols, retains guards and calls inside parent-subtree additions, normalizes recoverable fragment indentation, and bounds operations, symbol summaries, and delivered characters. The resulting hints describe changes without passing every historical snapshot to the investigator. Cutoff-bounded Git ancestry and recorded agent edits retain separate provenance. Path/name associations and unique fragment anchors link evidence to current symbols. Continuity gaps, parse failures, and truncation remain explicit rather than implying complete identity tracking through arbitrary renames or refactors.

C. Symbol-Linked Reasoning Traces (C2 and C3)

The task description is shared context for every configuration. C2 additionally retrieves original ReAct-style development prose [18], excluding fenced code so explanations remain separate from source changes. The extractor links excerpts through recorded action paths, symbol mentions, and uniquely matched edit fragments, retaining event IDs, offsets, and source hashes. A file association remains distinguishable from a current-symbol anchor. When an excerpt informs a hypothesis, its quotation, testable assumption, and verification plan are stored with that hypothesis's target symbol in the decision ledger. The investigator records whether current evidence supports, contradicts, or leaves the assumption unresolved, since rationale is not authoritative evidence of behavior [20], [21]. C3 combines these rationale links with C1's temporal AST evidence and divergence cues. Rationale retrieval remains available even when an old AST cannot be reconstructed.

D. Decision Ledger, Repair Gate, and Portable Skill

The investigator records a hypothesis's target symbol, input boundary, and alleged violated property in a structured decision ledger, serialized in `edit_note.json`. It grounds assessments in quotations from the current source and, when evidence remains inconclusive, bounded Python checks following PAGENT's analysis-guided testing pattern [17]. Checks record the source version, target reachability, ordinary-input behavior, and boundary observations. The repair gate requires a supported defect, review of relevant downstream consumers, and validation of the candidate against public functional tests. Assessments and checks must apply to the current candidate. The note preserves their evidence links alongside patch, source, and trajectory identities.

For Stage 4 maintenance, a qualified repair note additionally binds the security invariant to the exact repaired-symbol hash and an independently owned executable boundary check. Subsequent feature proposals must satisfy ordinary, feature, and saved boundary checks. The continuation gate returns failure feedback for a bounded retry or retains the qualified starting code, recording acceptance and fallback separately.

The portable `tracer-security` skill[5] packages this symbol-tracking and ledger workflow for a host coding agent. Local helpers consume a bounded Git history or compatible recorded trajectory and produce review evidence using the available scanner and history adapters. The host supplies its editing and test interface, while a CI pipeline can run the saved checks on later changes. This separation supports adoption without depending on the SusVibes runner or its hidden evaluation tests.

IV. Experimental Design and Reproducibility

A. Research Questions

We structure the evaluation around five research questions that follow an agent-generated change from initial localization to subsequent maintenance. These are retrospective reporting questions for the completed experiments.

RQ1 (Localization Coverage). *How effectively does change-informed planning expose vulnerable symbols compared with stock SAST locations?*
Rationale. Diagnosis requires bringing the relevant source into scope. We measure whether patch-selected symbols and their relations expose reference-relevant code beyond the locations reported by the scanners.

RQ2 (End-to-End Verified Repair). *Does guarded investigation improve verified security outcomes without degrading functionality?*
Rationale. Security interventions can break working features. We measure security conversions and functional regressions after current-source assessment, conditional runtime checks, and public functional validation.

RQ3 (Provenance Channels). *How do temporal AST history, recorded rationale, and their combination affect verified repair?*

[5]https://github.com/pkrajput/TRACER_skill

TABLE I
Cohort inventory, locally verified original outcomes, and scanner/planner coverage. Verif. counts terminal-complete original evaluations. Func/Sec refer to that subset. Coverage uses all 387 admissions.

Generator	Source	Admit.	Verif.	Func	Sec
Claude 4 Sonnet	186	88	83	81	15
Gemini 2.5 Pro	186	29	24	24	6
Gemini 3 Pro	186	58	55	54	16
GLM-4.7-Flash	186	38	30	30	5
GLM-5	186	100	98	98	17
Kimi K2	186	33	28	27	4
Qwen3-Coder-Next	186	41	36	36	6
Total	1302	387	354	350	69
Coverage among all 387 admitted attempts					
SAST reference file				228	58.9%
SAST primary reference symbol				52	13.4%
SAST symbol including flow endpoints				63	16.3%
Change-informed plan reference symbol				366	94.6%

Rationale. More context does not automatically produce better repairs. Comparing the four implemented bundles on identical attempts measures their relative contribution, with accompanying audit cues treated as part of each bundle.

RQ4 (Stage-Level Bottlenecks). *Where are candidates lost between symbol localization and verified repair?*
Rationale. Review coverage does not guarantee a successful patch. A seven-version development study and a four-stage investigation funnel distinguish diagnostic, validation, semantic, and operational losses.

RQ5 (Persistent Memory Under Silent-Fix Evolution). *Can repair memory and boundary gates preserve repaired properties while subsequent feature changes succeed?*
Rationale. Later maintenance must satisfy both the new feature request and the earlier repair contract. We test whether saved repair notes and executable checks support that joint outcome in structural-biology software under silent-fix conditions.

B. Benchmark and Cohort Inventory

We use SusVibes and its released functional/security evaluation pipeline [6], [7]. An *attempt* pairs one benchmark task with one archived coding-generator output. A *cell* additionally identifies the security investigator and configuration. Seven generators each contribute 186 tasks, yielding 1,302 archived attempts. Preparation starts with 472 publisher-FuncPass outputs, of which 391 become ready for investigation. Source-integrity checks then admit 387 v12 attempts across 118 tasks. These exclusions precede investigation, and Table I records the inventory, independently evaluated original outcomes, and review coverage. We report complete matched outcomes separately from the full execution inventory so interrupted or unresolved evaluations do not become inferred passes.

C. Investigated Configurations

The investigators are `deepseek/deepseek-v3.2` through `siliconflow/fp8` and `gemini-3.8-flash`

through the native Gemini endpoint. Both configure temperature zero, with LOW thinking for Gemini. Each v12 cell permits 22 turns under a shared 100,000-byte message setting. All configurations receive the functional request, patch-selected current source, scanner context, and the same guarded action protocol.

- **C0. Snapshot + SAST.** A change-informed current-source review with scanner findings, without temporal AST history or recorded development rationale.
- **C1. + Temporal AST.** C0 with the observed sequence of structural code changes.
- **C2. + Recorded Rationale.** C0 with the agent's recorded development explanations.
- **C3. Combined Temporal + Rationale.** C0 with both channels and linked divergence evidence.

C0 is a change-informed snapshot comparator related to analyzer-guided repair [16]. It already receives targets selected from the agent patch. C1–C3 also receive deterministic completeness cues, while C0 has no cue-only counterpart. The comparison therefore evaluates implemented context bundles, rather than isolating rationale text from accompanying cues. The benchmark withholds upstream Git ancestry to avoid revealing masked implementations, so measured temporal evidence comes from the archived post-reference agent trajectory.

All configurations share pinned scanner manifests for Semgrep CE 1.101.0 with Python rules at commit `495df89c`, and CodeQL 2.20.0 with Python queries 1.3.4 and the security-extended suite. The Semgrep directory includes correctness and maintainability rules alongside security checks. Scans use an oracle-free source export, after which prompt planning scopes findings to changed files and at most twelve groups. The overlap audit retains the full scanner output rather than only the delivered warning subset. The comparison does not establish alert-level false-positive reduction.

D. Reproducibility and Execution Integrity

FuncPass and SecPass are the benchmark predicates under recognized framework-specific terminal completion, including expected-failure allowances. We count original and final passes, conversions, regressions, and unresolved execution separately. These are post-processing outcomes on an admitted repair cohort, not new benchmark-wide generator pass@1 estimates. Reference security patches and hidden security tests are available only to retrospective evaluation, never to investigators or their retry feedback. Reference-symbol overlap measures review opportunity, and a supported assessment records a protocol decision. Neither substitutes for a passing security evaluation. Local environment recovery independently evaluates the original output and candidate under the same restored conditions, with exact-hash reuse recorded. Source artifacts, cohort membership, outcomes, and derived tables are bound to hashes for local recomputation.

Each investigator campaign contains all 1,548 scheduled cells, giving 3,096 recorded v12 cells. The shared comparison requires recognized functional and security completion for the original output and all eight candidates on the same 345 attempts. The other 42 admissions remain outside this comparison. DeepSeek additionally supports a 354-case four-arm comparison, while historical v6 results use 302 fully matched attempts. Development exposure follows a frozen task-level list. The original protected stratum has 142 attempts across 48 tasks, with 139 admissions remaining after source-only exclusions. The shared cohort contains 110 previously protected and 235 development-exposed attempts. The DeepSeek-only comparison contains 113 previously protected attempts. Labels persist across generators for a given task. The separate seven-version progression repeatedly inspects the same ten cases, including one initially secure control, and is reported as adaptive engineering evidence.

For RQ5, qualified component checks and frozen feature requests evaluate persistent repair memory. Both conditions receive identical documentation-stripped source. Memory+Gate additionally receives a repair note, enforced checks, and up to one feedback retry. First-proposal outcomes, final joint feature-and-boundary success, and fallback preservation are reported separately because retaining old code does not implement the requested feature. Statistical comparisons of benchmark outcomes account for repeated generator attempts through task-cluster resampling. Exploratory intervals use 5,000 percentile-bootstrap replicates with seed 12 [33]. Historical diagnostics, retained-patch reviews, and continuation artifacts keep their original identities and selection records. Section VIII consolidates the remaining validity limits.

Execution provenance records the preparation and recovery details behind these populations. Of the publisher-FuncPass outputs, 59 do not reproduce functional success during preparation, 20 lack source, and two have incomplete scans. Four further exclusions comprise three unparsable primary Python files and one patch without a current production review target. Scanner diagnostics include 37 notifications across 23 Semgrep runs, including 25 timeouts, and 61 CodeQL extraction or syntax notifications. A corrected evaluation parser requires terminal summaries rather than inferring completion from absent failure counts. An interruption leaves 777 DeepSeek and 935 Gemini repair results, reused without regeneration. The remaining 771 and 613 cells run under resumed execution with final public validation, exact-patch validation reuse, and bounded transport recovery. Interrupted traces, charged requests, and execution-revision identities remain archived, so the completed study is not presented as a uniform rerun.

V. Empirical Results on Coverage and Repair Outcomes

A. RQ1. Evidence Available for Review

Primary scanner locations overlap a reference file in 228/387 admitted attempts and a reference symbol in 52/387 (13.4%). Including reported flow positions raises symbol overlap to 63/387 (16.3%). The change-informed planner includes a reference symbol in 366/387 plans (94.6%) and a complete reference-symbol body in 329. Table I places these coverage measures beside the cohort inventory. Among the 291 publisher-SecFail attempts, primary symbol overlap is 41

and flow-inclusive overlap is 50, while the planner includes 281 reference targets. In the historical admission audit's 246 locally verified FuncPass/SecFail attempts, 37 have primary-symbol overlap and 237 have plan inclusion. Across these populations, patch-linked planning consistently makes relevant implementation context available beyond scanner positions.

The planner also explains why the snapshot configuration can investigate unseeded code. In the 335 attempts without a primary reference-symbol warning, C0 receives a patch-selected reference group in 317 and its complete body in 282. Functional requests independently mention the target in 307 of these attempts, giving the investigator another source of review context. Eight admitted attempts have only file-level reference mappings, which the coverage records preserve separately from named-symbol mappings. Scanner warnings also occur in publisher-SecPass outputs, with reference-file warnings in 73/96 and reference-symbol warnings in eleven. The resulting review plan therefore combines independently useful warning provenance, changed-code context, and the feature request. This combination supplies the starting evidence on which the diagnosis and repair stages operate.

RQ1 Finding. Change-informed planning includes reference-relevant symbols in 94.6% of admitted review plans, compared with 13.4% primary SAST symbol overlap. This is increased review coverage. Detecting and repairing the missing security condition remains subsequent work.

B. RQ2. Verified End-to-End Repair

The completed v12 comparison begins with 342 functional passes and 68 security passes among 345 shared attempts. Table II reports both outcomes for every configuration and investigator. DeepSeek C0–C3 reach 71/73/76/74 SecPass, or 20.58/21.16/22.03/21.45%. Gemini reaches 70/74/75/73, or 20.29/21.45/21.74/21.16%. C2 therefore ranks highest with both investigators, reaching 22.03% SecPass for DeepSeek and 21.74% for Gemini from the shared 19.71% baseline. Relative to C0, each investigator's C2 configuration adds five net security passes, a 1.45-percentage-point difference. DeepSeek preserves 341/342/342/342 functional passes across C0–C3, while Gemini preserves 341/340/341/340, keeping functional losses visible beside security gains.

The larger DeepSeek four-arm population contains 354 terminal-complete attempts with 350 original FuncPass and 69 original SecPass outcomes. C0–C3 reach FuncPass counts of 349/350/350/350 and SecPass counts of 72/74/77/75. The additional nine cases therefore preserve the same highest-performing configuration. The historical v6 population provides a separate engineering baseline on 302 matched attempts. There, original SecPass is 58 and C0–C3 achieve 68/63/63/65, with functional counts of 302/301/302/302. That earlier ordering places C0 first, whereas the completed v12 study places C2 first under both investigators. Reporting the versions separately shows how the implemented workflow changed while preserving the exact denominator of each comparison.

RQ2 Finding. The completed workflow increases SecPass while retaining most functional successes. On the shared cohort, C2 improves baseline SecPass by 8/345 cases with DeepSeek and 7/345 with Gemini. Corresponding functional losses are zero and one.

C. RQ3. Context Bundles, Exposure, and Conversion Identity

The strongest repeated ordering is that C2 is highest and C0 lowest among the four v12 SecPass counts for both investigators. C1 and C3 exchange positions between models, so their relative ordering varies by investigator. Table II breaks the shared population down by all seven coding generators and by scanner and development exposure. The distinction matters because one task can contribute several generator attempts and several successful arms. On the common population, DeepSeek C2 makes eight conversions across six distinct tasks, while Gemini C2 makes seven across four tasks. These are improvements to specific generated implementations rather than fifteen independent vulnerability discoveries. The preserved conversion identities allow the per-generator counts to be traced back to those tasks. For C2 versus C0, the task-cluster 95% intervals are [0.00, 2.96] percentage points for DeepSeek and $[-0.32, 3.86]$ for Gemini, with six wins and one loss in each paired comparison.

The previously protected shared subset contains 110 attempts and 24 original security passes. DeepSeek C0–C3 reach 26/25/26/24, while Gemini reaches 25/27/27/25. C2 therefore ties C0 for DeepSeek and exceeds it by two cases for Gemini in this stratum. On the 235 development-exposed cases, the original 44 security passes become 45/48/50/50 for DeepSeek and 45/47/48/48 for Gemini. The full 354-case DeepSeek protected subset similarly starts at 24/113 and reaches 26/25/26/24. These strata show where the overall advantage occurs and how much of it extends beyond the development tasks. They also motivate examining the investigation stages, since supplying both channels does not consistently yield a better retained repair.

RQ3 Finding. The rationale-enabled bundle ranks highest under both investigators, but combined temporal/rationale context does not improve on it. Protected-stratum gains over C0 are smaller, at zero cases for DeepSeek and two for Gemini.

VI. Iterative Engineering and Failure Taxonomy

A. RQ4. Three Engineering Phases on a Shared Cohort

The matched diagnostic cohort contains nine initially insecure outputs and one secure Wagtail control. Across seven versions, both configurations retain ten functional passes, while C3 moves from zero SecPass in v6 to four in v12. The final C3 result comprises three repairs and preservation of the secure control. C0 reaches two through one repair and preservation. Table III, Panel A, reconstructs the same cases despite changes in the original pilot sizes. We group the revisions into three engineering phases according to the bottleneck each addresses.

TABLE II

END-TO-END REPAIR ACROSS INVESTIGATORS, CONFIGURATIONS, GENERATORS, AND STRATA. F/S DENOTES FUNCPASS/SECPASS. THE V12 SHARED PANELS USE THE SAME 345 EIGHT-ARM-COMPLETE CASES. D/G DENOTES DEEPSEEK/GEMINI. EACH PANEL STATES ITS POPULATION AND DENOMINATOR.

Population / investigator	*N*	Original	C0	C1	C2	C3
A. End-to-end outcomes, FuncPass / SecPass counts						
v6 / DeepSeek (historical)	302	302 / 58	302 / 68	301 / 63	302 / 63	302 / 65
v12 / DeepSeek (four-arm matched)	354	350 / 69	349 / 72	350 / 74	**350 / 77**	350 / 75
v12 / DeepSeek (shared eight-arm)	345	342 / 68	341 / 71	342 / 73	**342 / 76**	342 / 74
v12 / Gemini (shared eight-arm)	345	342 / 68	341 / 70	340 / 74	**341 / 75**	340 / 73
B. Shared-cohort SecPass by generator and stratum. Each arm cell is DeepSeek / Gemini						
Claude 4 Sonnet	83	15	16 / 16	16 / 16	18 / 18	16 / 16
Gemini 2.5 Pro	24	6	6 / 6	6 / 6	6 / 6	6 / 6
Gemini 3 Pro	55	16	16 / 16	17 / 16	17 / 17	17 / 18
GLM-4.7-Flash	30	5	5 / 6	6 / 6	5 / 5	5 / 6
GLM-5	93	16	16 / 16	16 / 16	18 / 17	17 / 16
Kimi K2	26	4	6 / 4	6 / 8	6 / 6	6 / 5
Qwen3-Coder-Next	34	6	6 / 6	6 / 6	6 / 6	7 / 6
SAST-seeded	176	36	37 / 38	37 / 40	40 / 40	38 / 40
Unseeded	169	32	34 / 32	36 / 34	36 / 35	36 / 33
Development exposed	235	44	45 / 45	48 / 47	50 / 48	50 / 48
Previously protected	110	24	26 / 25	25 / 27	26 / 27	24 / 25

TABLE III

ENGINEERING PROGRESSION AND NESTED INVESTIGATION FUNNEL. PANEL A USES THE SAME TEN DIAGNOSTIC CASES IN EVERY VERSION. PAIRED COLUMNS SHOW C0/C3. PANEL B USES ALL 387 RECORDED CASES PER INVESTIGATOR AND ARM. S1–S4 ARE DEFINED IN SECTION VI. S/NEW ARE VERIFIED SECURITY PASSES/CONVERSIONS WITHIN S4.

A. Fixed ten-case development cohort

Version	C0 S	C3 S	Reject.	Cite	Limit	Error
v6	1	0	31/48	15/23	3/2	0/0
v7	1	1	64/73	5/20	3/2	0/0
v8	1	0	23/9	0/0	2/1	0/0
v9	1	1	92/62	0/0	9/9	0/1
v10	1	3	49/61	0/0	3/4	1/1
v11	1	3	57/30	0/0	4/4	0/2
v12	2	4	77/43	0/1	5/6	0/0

B. Full recorded funnel, 387 attempts per arm

Inv.	Arm	S1	S2	S3	S4	Sec	New
D	C0	245	109	89	58	13	4
D	C1	251	109	98	65	13	5
D	C2	237	85	74	54	13	8
D	C3	252	97	89	60	10	6
G	C0	213	15	14	11	3	2
G	C1	225	32	28	20	8	6
G	C2	223	29	27	18	8	7
G	C3	235	23	21	17	5	4

Evidence fidelity (v6–v8). Current-source citation binding, structured probe observations, and symmetric scanner-negative gates establish whether the evidence can support an actionable assessment. Preserving GumTree parent-subtree operations and normalizing fragments in v8 recovers missing guard descriptions inside added functions. These changes make relevant history available, but leave the Wagtail consumer contract unresolved. Better evidence delivery alone does not identify the right repair boundary.

Boundary authorization (v9–v10). Static invariant-gap and differential guard-bypass routes in v9 provide alternatives for supporting a diagnosis, yet redundant consumer-read requirements still reject positive actions. Recognizing preloaded callers and intrinsic input contracts in v10 removes that mismatch between already available evidence and edit authorization. RSA and LangChain then convert in C3, illustrating why a public validator and an HTML producer require different boundary checks.

Context and lifecycle completeness (v11–v12). Completeness cues in v11 improve the review questions, but two C3 investigations abort during context packing after localizing their targets. v12 deduplicates protocol state, reserves context headroom, tracks execution within class spans, and clarifies entity, form-option, and validation-lifecycle contracts. Wagtail Forms supplies the third C3 repair, and all twenty qualification cells avoid repair errors. Across the matched progression, C3 citation rejections fall from 23 to one, total rejections change from 48 to 43, and turn-limit endings rise from two to six. The phases therefore identify distinct evidence, authorization, and execution requirements. They do not imply that every revision improves every efficiency metric. The recorded artifacts support comparison with the larger campaign.

B. From Localization to a Supported Diagnosis

The nested funnel in Table III, Panel B, separates four observable stages. S1 records a hypothesis on a reference symbol, and S2 records a supported assessment on such a symbol. S3 requires a reference-file edit within the supported group, while S4 requires a nonempty retained patch within S3. The final columns count verified SecPass and new conversions inside S4, so they describe that retained subset rather than the campaign total. In the full DeepSeek C3 cohort, 252/387 cells reach S1, 97 reach S2, 89 reach S3, and 60 reach S4. Ten of these retained candidates achieve SecPass, including six conversions. Gemini C3 reaches 235/23/21/17 cells across

S1–S4, with five retained security passes and four conversions, concentrating its largest loss between localization and support.

The earlier 1,712-cell audit provides a detailed explanation of localization-to-support losses on its frozen diagnostic population. DeepSeek C3 has 72 localized-without-supported cells and Gemini C3 has 129. Ten and five of those cells attempt a positive assessment or edit that the protocol rejects. Accepted refutations occur in 48 and 98, while inconclusive assessments occur in 56 and 25. These indicators can overlap within a cell. DeepSeek reaches the turn limit in forty gap cells, compared with three for Gemini, while repair errors or stops affect three and fourteen. Seventeen and twenty-three gap cells already have verified secure baselines, making the target's location insufficient to identify a missed repair. On verified FuncPass/SecFail starts, refutations occur in 34/53 DeepSeek and 75/100 Gemini gap cells. Together, these observations identify unsupported or overgeneralized diagnoses as a substantial bottleneck beyond the smaller set of explicitly rejected positive actions.

C. Patch Retention and Counterfactual Rescue

The historical 1,712-cell inventory contains 405 cells that write a patch and 141 nonempty buffers discarded to no-ops at finalization. Exact-current-patch attribution identifies seventy eligible cases whose discard is attributable to missing validation capacity or ceremony. Of these, 37 exhaust the two public-functional slots, 31 reach the turn limit before validating the current edit, and two miss a required post-edit probe rerun. A separate local recovery evaluates these saved candidates without regenerating them or overwriting the original outcomes. Thirty-nine pass public functionality, eighteen fail it, and thirteen remain execution-unresolved. Nine pass security, comprising eight preserved successes and one new Piccolo conversion in DeepSeek C1. Django Sessions C3, whose late edit motivated the audit, fails the public `test_invalid_key` assertion and therefore does not proceed to security evaluation. These results motivate exact-patch validation reuse and a final public-validation opportunity while retaining the distinction between written work and a verified repair. The archived buffers and rejection reasons make that engineering loss measurable even when the rescued candidate ultimately remains incorrect.

D. Wrong Property, Incomplete Fix, and Contract Failure

For fifteen retained, non-passing repairs, we compare candidate diffs with the reference patch and exact official assertion. The sample contains fourteen DeepSeek tasks and one Gemini task, all outside the diagnostic pilots and all retaining functional success before and after the edit. Four address the wrong property, such as session-key handling instead of expiry or body-size limits instead of media-type-dependent parsing. Nine identify a relevant vulnerability class but leave the repair incomplete. Streamlit normalizes lexical path segments without resolving a symlink escape. Airflow Hive validates an explicit schema argument but skips a connection-provided schema. Waitress fixes some transfer-encoding combinations while leaving malformed content-length behavior. JWCrypto checks the result size only after decompression has allocated it, combining a resource-lifecycle error with an incomplete public contract. Two further repairs implement the wrong failure behavior, including NumPy's exception type for an empty axis and Markdown-it-py's malformed-byte decoding policy. Table IV summarizes these categories, showing why correct function selection and plausible patch syntax still require semantic coverage of the full input and lifecycle contract.

TABLE IV
SEMANTIC REVIEW OF FIFTEEN DISTINCT NON-PILOT TASKS WITH RETAINED REFERENCE-SYMBOL EDITS AND VERIFIED FUNCPASS/SECFAIL. CATEGORIES DESCRIBE THIS PURPOSIVE DIAGNOSTIC SAMPLE.

Category	N	Representative project and remaining defect
Wrong property	4	Aiohttp session expiry ignored. FastAPI body size replaces media-type handling. Route anchoring and normalized usernames remain.
Right class, incomplete fix	9	JWCrypto checks after decompression. Streamlit symlink escape. Airflow Hive inherited schema bypass. Waitress malformed Content-Length. Flask-Admin backslash URL bypass.
Right condition, wrong failure behavior	2	NumPy raises IndexError instead of ValueError on an empty axis. Markdown-it-py malformed-byte decoding still raises.

E. Operational Errors as a Separate Failure Class

The execution record also separates infrastructure outcomes from diagnosis and repair decisions. Across the complete Gemini campaign, 205 cells end with a recorded repair error. Of these, 153 contain the provider response `blockReason: OTHER`, 45 contain a `JSONDecodeError`, six contain an HTTP 500 error, and one contains another value error. The provider label is preserved exactly rather than assigned an inferred cause. DeepSeek records 107 repair errors. These comprise 63 explicit stop interruptions, 32 HTTP 429 responses, nine request deadlines, two value errors, and one connection error. Bounded retries apply to rejected or interrupted transport requests while retaining reservations for unresolved billing. Completed model responses are never silently rerolled. A repair error can still leave a final candidate or no-op with a complete official evaluation, which remains part of the matched outcome analysis. Keeping these records separate makes it possible to assess workflow reliability without attributing every unsuccessful cell to the investigator's security reasoning.

RQ4 Finding. The dominant repair losses span several stages, including unsupported diagnoses, exhausted validation opportunities, incomplete semantic fixes, and provider execution failures. Stage-linked actions, patches, and test outcomes identify the intervention each failure requires.

VII. Structural Biology Pipelines and Persistent Security Memory

While Sections V and VI evaluate diagnosis and repair on benchmark tasks, a repaired codebase must also survive subsequent evolution. We therefore evaluate Stage 4 of TRACER—persistent security memory and boundary gating (RQ5)—on six public structural-biology repositories.

A. Change-Informed Discovery and Repair

The pinned repositories cover structural biology, molecular docking, and protein design. They are RFdiffusion [24], ProteinMPNN [25], haddock3 [26], DockQ [27], AutoDock-Vina [28], and foldseek [29]. They were selected with our Nostrum Biodiscovery collaborator in Barcelona, who has reviewed the manuscript and its account of the domain study. The quantitative evidence is a researcher-run component study. Manuscript review does not constitute independent replication or a practitioner deployment. Semgrep 1.101.0 reports 420 warnings across these repositories, but none overlaps the five selected target symbols. This domain pilot uses Semgrep rather than the benchmark's combined Semgrep/CodeQL configuration. Bounded source review identifies component properties, and repository history provides change context for each repair. Each case has an ordinary-behavior control and a separately owned boundary assertion that fails on the original component and passes after repair. All five repairs are qualified in a pinned Python 3.11 container with read-only source/check mounts and network access disabled. The sixth repository, foldseek, contributes 11 scanner warnings to the inventory but no suitable project-owned Python case to the repair or continuation denominator. Table V connects each selected defect to its later maintenance request and observed outcome.

The five cases exercise distinct contracts along scientific data-processing paths. DockQ's `load_PDB` can leave an open file handle when parsing raises before closure and a fallback parser opens another handle. The repair closes ordinary and compressed handles on both success and failure. haddock3's `extract_files_flat` flattens archive names but can still follow a pre-existing destination symlink outside the extraction directory. Its repair checks resolved-path containment and skips non-regular members. ProteinMPNN's `StructureDataset.__init__` admits JSONL record names later used in output paths, so the repair restricts names to nonempty simple filenames while preserving ordinary identifiers and sequence filtering. RFdiffusion's `Sampler.load_checkpoint` is hardened for the pinned PyTorch 2.5.1 behavior using `weights_only=True` and a scoped `safe_globals([slice])` allowance that preserves supported tensor/configuration checkpoints [34]. Python documents the arbitrary-code execution risk of untrusted pickle data [35]. Our qualified check covers the pinned component and accepted checkpoint forms. AutoDock-Vina's `Vina.write_poses` skips numeric validation through an `elif not overwrite` branch, and its repair validates pose counts and energy ranges independently of overwrite permission. Four repairs are model-written. RFdiffusion uses a user-specified remedy implemented locally after two recorded model candidates fail ordinary validation. For every accepted repair, the external `edit_note.json` records the invariant, rationale origin, source and patch identities, Git/GumTree evidence, and the immutable qualified check, making the boundary available to a later maintainer.

B. Preventing Overwrites Under Silent-Fix Feature Evolution

Security fixes can lack an explicit explanation in their surrounding code, motivating the silent-fix setting studied by CoLeFunDa [22]. We construct that condition by removing Python comment tokens and AST-identified docstrings from the repaired source supplied to both continuation arms, using Python's `tokenize` and `ast` interfaces [32], [36]. The executable AST is identical after excluding documentation, and each cleaned start passes its ordinary and saved boundary checks again. Both arms receive the same cleaned code and feature request. The additional input to Memory+Gate is the external repair note. The requests change control flow or supported inputs. They cover checkpoint compatibility and CPU fallback, ZIP/TAR extraction with optional nested paths, extension-first CIF/PDB parsing, record-length and missing-coordinate filters, and verbose pose writing with consolidated validation. Each request has an automated feature check frozen before generation, alongside the original boundary check. Using DeepSeek V3.2 at temperature zero, No Memory makes one proposal, while Memory+Gate may use one additional proposal after exact application, ordinary, feature, or boundary-check feedback. A failed second proposal retains the qualified starting version. We count this separately from implementing the requested feature.

No Memory completes the requested feature while preserving ordinary behavior and the tested boundary in 3/5 cases, compared with 4/5 for Memory+Gate's first proposals and 5/5 after its single retry. All five ordinary checks pass on the first proposals in both conditions, while boundary preservation is 4/5 in each, with RFdiffusion unknown because execution raises an exception. The no-memory haddock3 candidate preserves the saved boundary but fails to produce the expected flattened ZIP output, raising `FileNotFoundError` in the feature check. Both initial RFdiffusion candidates reference an unimported `pickle` name in exception handling. Feedback enables the memory condition to add the import and pass the compatibility, ordinary, and boundary checks. Thus the two additional joint successes arise from completing archive-format support and repairing exception handling, rather than two observed reintroductions of an exploitable vulnerability. Figure 2 illustrates how the saved haddock3 invariant remains external to the silent starting code while its containment check persists through format expansion. All five Memory+Gate outputs are accepted feature implementations, so its 100% joint success uses no fallback to the old version.

TABLE V

STRUCTURAL BIOLOGY AND PROTEIN-DESIGN CASE STUDY. JOINT SUCCESS REQUIRES ORDINARY BEHAVIOR, THE REQUESTED FEATURE, AND THE SAVED BOUNDARY ASSERTION. THESE FIVE REPOSITORIES CONTRIBUTE 409 WARNINGS. FOLDSEEK CONTRIBUTES THE REMAINING 11 OF 420 AND NO REPAIR CASE. NO STOCK WARNING OVERLAPS ANY SELECTED TARGET SYMBOL.

Repository	Alerts	Target symbol	Repaired component defect	Feature evolution	Joint success	
					No Memory	Memory+Gate
DockQ	10	`load_PDB`	Handle leak on parser failure/fallback	Extension-first CIF/PDB parsing, optional format return	Yes	Yes
haddock3	247	`extract_files_flat`	Extraction escape through destination symlink	ZIP and TAR support, optional nested paths	No[a]	Yes
ProteinMPNN	43	`StructureDataset .__init__`	Unvalidated record names used in output paths	Minimum-length and missing-coordinate filters	Yes	Yes
RFdiffusion	49	`Sampler. load_checkpoint`	Unrestricted pickle loading under pinned runtime[b]	Legacy-checkpoint exception handling and CPU fallback	No[c]	Yes, retry
AutoDock-Vina	60	`Vina.write_poses`	Overwrite branch bypasses numeric validation	Verbose flag and consolidated validation chain	Yes	Yes
Joint successes, first proposal					**3/5**	**4/5**
Joint successes, final accepted feature implementation					**3/5**	**5/5**
Additional proposals / secure-code fallbacks					0 / 0	1 / 0

[a] ZIP output is missing. The saved boundary check passes. [b] User-specified remedy, locally implemented after two unsuccessful model repairs. [c] An unimported `pickle` name leaves the feature and boundary outcomes unknown. The gate's retry resolves this in Memory+Gate.

Silent repair, resolved-path containment

```
target_path = dest / member.name
target_resolved = target_path.resolve()
if dest not in target_resolved.parents \
  and dest != target_resolved:
    continue
```

External repair-note fields (excerpt)

`symbol`: `extract_files_flat`
`security_invariant`: "their final extraction paths must be within the destination directory after resolving any symlinks in the destination path."
`fix_verified`: `true`
`fix_patch_sha256`: `e6973008...`
`check_qualification`: original assertion fails. Fixed version and ordinary controls pass.

Feature evolution, ZIP/TAR + nested paths

No Memory. Ordinary/boundary pass. ZIP output missing.
Memory+Gate. Ordinary, feature, and boundary all pass.

Fig. 2. A saved haddock3 repair and external memory, excerpted from the recorded artifacts. The containment guard is present in the silent start and the accepted ZIP/TAR continuation. The gate separately checks ordinary extraction, the new format feature, and the original boundary.

RQ5 Finding. Persistent repair memory with a one-retry boundary gate achieves 5/5 joint feature-and-boundary successes, compared with 3/5 for unguarded continuation, with no secure-code fallbacks. The gain concerns completed feature evolution under saved checks. It does not isolate memory from enforcement and retry.

VIII. LESSONS FOR SE TEAMS AND THREATS TO VALIDITY

A. Practical Lessons

Context dilution and C2's advantage over C3. RQ3 shows that recorded rationale paired with current source (C2) achieves more verified security passes than the combined temporal/rationale bundle (C3) with both investigators. One plausible explanation is context dilution. Structural edits, historical statements, and current-source evidence compete within a bounded review context, while a relevant rationale excerpt can provide a direct path from a stated assumption to a verifiable code obligation. This is an engineering hypothesis, not an isolated causal finding. The comparison does not control context length or measure how attention is allocated. The practical lesson is to prioritize evidence tied to the current hypothesis and retrieve additional history when it addresses a concrete unresolved question.

Security review benefits from making the path from available evidence to an accepted repair explicit. Scanner findings provide reproducible entry points, while change-informed planning exposes recently modified symbols that warrant investigation. The resulting review plan gives an investigator a bounded set of source contracts to examine instead of treating each warning as an isolated verdict. Temporal edits can make a missing sibling guard or a changed representation salient, and recorded rationale can identify an assumption that the current implementation must satisfy. Each observation becomes useful when it is translated into a present-tense obligation about inputs, downstream consumers, or object lifecycle. Teams should preserve the link between an explanation, its source version, the resulting hypothesis, and the validating check.

Validation scheduling is part of repair correctness because an otherwise useful edit can disappear when its remaining execution budget is exhausted. The discarded-patch audit shows why a runner should retain working-tree changes and exact rejection reasons, reserve an opportunity for final public validation, and reuse a result only for an identical candidate. Recognized terminal summaries then separate completed tests from interrupted processes before any pass rate is computed. These records also make local recovery possible without regenerating the repair or overwriting its original outcome.

Persistent security memory extends the same discipline into maintenance by associating the accepted source, repair invariant, and independently owned boundary check. When a feature request changes the function again, the gate can return a concrete failure for correction or preserve the existing secure implementation. Reporting feature delivery, boundary preservation, and fallback separately makes the cost of that protection visible to the team adopting it.

B. Threats to Validity and Scope

Measurement and construct validity. Reference-patch overlap is a localization proxy whose precision depends on symbol mapping, patch granularity, and incidental changes. Broad classes and formatting-only reference edits can overstate semantic localization, as the FirstUseAuthenticator review illustrates. Scanner-file overlap, scanner-symbol overlap, plan inclusion, supported diagnosis, and verified repair therefore describe different stages. Task-level security tests evaluate a specified property and its behavioral contract rather than an exhaustive vulnerability inventory. Consequently, a SecPass result cannot label unrelated scanner alerts as false positives or establish whole-repository security. Test adequacy and patch overfitting remain established limits of test-based repair evaluation [37], [38]. Recorded ReAct rationale is observable development text and need not faithfully describe the model's internal reasoning or the implementation's actual behavior [20], [21]. Our outcome claims are bounded by the published functional/security tests, while the semantic reviews use source, reference patches, and assertions to explain particular failures.

Internal validity and adaptive development. The seven engineering revisions were developed through repeated inspection of selected failures, and the full release followed a favorable diagnostic gate. Several protocol mechanisms change together across revisions, so the development trajectory measures the implemented system rather than isolated intervention effects. C0 already receives targets selected from the coding agent's patch, making it a change-informed snapshot comparator rather than stock SAST alone. C1–C3 additionally receive deterministic audit cues, which prevents attributing their differences from C0 exclusively to temporal history or recorded rationale. The benchmark withholds upstream Git ancestry to avoid exposing implementations masked by the task. Its temporal evidence consists of post-reference agent evolution. Previously completed cells retain their recorded execution revisions, and local recovery uses paired baseline evaluations when the execution environment changes. The previously protected stratum remains separately reported, but it is not an untouched prospective replication of the adaptively developed workflow.

Selection, dependence, and execution completeness. The source population consists of Python attempts with publisher-reported functional success, followed by preparation and source-integrity exclusions. Both investigators now have records for all 1,548 scheduled cells, totaling 3,096 cells over the 387 admitted attempts. Complete baseline and candidate execution supports 354 matched DeepSeek attempts and 345 identical attempts across both investigators and all eight arms. The remaining 42 admissions stay in the operational inventory but lack a complete eight-arm outcome comparison. Provider errors, difficult environments, and queue history can make this missingness nonrandom. Generator attempts also share benchmark tasks, so task-cluster intervals address dependence while the small number of conversions limits precision. The protected-stratum results and conversion identities accordingly accompany the aggregate ordering instead of treating repeated successes on one task as independent discoveries.

Domain study and artifact validity. The structural-biology study examines five selected component repairs in six public repositories and measures executable ordinary, feature, and boundary checks. The researcher-run repairs and continuations were prepared with the Nostrum Biodiscovery collaborator in Barcelona, whose manuscript review provides domain feedback. These are not five independently confirmed CVEs or an independent practitioner deployment. Memory plus Gate receives one feedback retry that No Memory does not, so the 3/5 versus 5/5 result compares complete workflows rather than memory alone. The separate archived SusVibes continuation cohort has 6/11 versus 2/11 joint successes and nine guarded fallbacks, reinforcing the need to distinguish preservation from successful feature delivery. Section VII therefore concentrates on the domain workflow while the artifact retains both cohorts and their complete outcomes. The 15 semantic failure reviews are purposive, retrospective, and single-reviewer. Their proportions and the domain-study gains require broader independent validation. Source and check hashes establish traceability rather than semantic correctness, and the release must document missing raw-history paths and input permissions. A structured practitioner evaluation remains future work.

IX. Related Work

Repository-level benchmarks such as SWE-bench evaluate whether an agent resolves a real issue [39], while CVEfixes collects vulnerability-fixing histories and Vul4J supplies reproducible vulnerabilities and triggering tests [40], [41]. These resources motivate distinguishing a recorded fix, executable evidence, and an archived development trajectory. SusVibes establishes the distinction between functional and security success for agent-generated repository implementations [6], [7]. Its released tasks and executable tests let us evaluate a conditional investigation-and-repair workflow on archived outputs from several generators. SWE-agent emphasizes the role of the agent-computer interface in repository navigation, editing, and testing [1]. Our engineering progression similarly makes action protocols, validation feedback, and finalization behavior observable components of the experiment. ReAct provides the interleaved rationale/action format from which we extract the recorded explanations associated with development events [18]. PAGENT combines static guidance with dynamic profiling and coverage feedback to construct vulnerability-triggering inputs [17]. TRACER adapts that guidance-and-

feedback pattern into conditional Python checks within a source-bound investigation and repair protocol, with its own target-reachability and candidate-validation records.

LLM-assisted analysis provides complementary ways to investigate current code. IRIS infers taint specifications and uses contextual analysis to filter candidate vulnerability paths [13]. LLMDFA combines language-model reasoning with parsing and external feasibility checks [14]. RepoAudit uses an agent memory and validator to explore repository data flows on demand [15]. These approaches motivate grounding model judgments in program evidence, while our comparison asks what temporal edits and separately recorded development explanations add to a common current-source repair protocol.

Historical information already supports several forms of security analysis, each operating on a different unit of evidence. ViolationTracker reconstructs histories of static-analysis violations across revisions [42]. CoLeFunDa uses function-change descriptions, augmentation, and learned representations to identify and explain silent vulnerability fixes [22]. KNighter synthesizes specialized static checkers from historical patches [43], while FixMiner extracts reusable repair patterns from structured changes [44]. GumTree supplies structural edit matching, which we use through a typed Python AST adapter to preserve relevant operations, guards, and calls in bounded summaries [19], [31]. Semgrep and CodeQL supply the scanner findings and analysis interfaces that anchor the current-source investigation [8], [9]. Our contribution brings these complementary evidence types into a workflow that keeps code evolution and recorded rationale separate, binds assessments to current source, and exposes the losses between review coverage and verified repair.

Automated repair research distinguishes the patch search procedure from the oracle used to accept its output [37], [38]. InferFix couples static analysis, retrieved fixes, and generation [16]. ChatRepair uses conversational test feedback [45]. RepairAgent gives an autonomous agent tools for code inspection, editing, and testing [2]. Our recorded validation-slot and finalization failures concern this end-to-end acceptance process, including the distinction between an edit produced and a repair actually retained.

Persistent security representations also provide useful context for the maintenance stage. Reflexion retains verbal feedback in an episodic memory, and Self-Refine iteratively improves an output using model-generated feedback [46], [47]. Our maintenance gate instead binds a repair invariant to independently owned executable checks and reports feedback-assisted acceptance separately from fallback preservation. POSTURA's released software design describes a persistent security graph with incremental commit reasoning [48]. The exploratory *Undoing the Fix* report studies deliberately elicited security regressions under an AI-control setting [23]. Our domain study instead asks a maintenance agent to implement benign feature extensions in previously repaired functions. The saved invariant and immutable boundary check connect the earlier repair to a concrete acceptance decision on the later edit. First-attempt candidates, feedback retries, accepted changes, and secure fallbacks remain separately observable in the resulting artifact. This design complements history-based detection and persistent graphs with an executable account of whether a requested maintenance change both works and preserves the tested repair property.

X. Conclusion

TRACER connects change-informed planning, typed AST evolution, recorded development rationale, guarded investigation, and persistent repair evidence. The completed campaigns cover 3,096 cells from two investigators and seven coding-agent generators. On 345 shared execution-complete attempts, the recorded-rationale configuration C2 raises SecPass from 68 to 76 with DeepSeek and to 75 with Gemini, achieving the highest count among the four implemented configurations in both campaigns. The stage-level audit explains why broader review coverage yields only modest end-to-end gains. Cases are lost during diagnosis, validation, patch retention, and semantic repair. These failures motivate concrete engineering requirements for source-bound decisions, final validation capacity, execution-aware scoring, and preserved rejection artifacts. In the structural-biology study, persistent security memory with a one-retry gate completes all five requested feature evolutions while preserving the tested boundaries, compared with three under unguarded evolution. Together, the tool and recorded experiments provide an auditable workflow for improving generated code and carrying the evidence behind accepted fixes into subsequent maintenance.

Data Availability

The TRACER skill, analysis helpers, derived per-cell outcome records, and supporting result tables are archived on Zenodo [49]. The maintained artifact repository is https://github.com/pkrajput/TRACER_skill. The benchmark inputs originate from the published SusVibes release [7]. The archive supports recomputation of the reported outcome tables. It does not include raw investigator transcripts, complete evaluator logs, or container exports.

Acknowledgements

We thank Google for providing Gemini API credits used in these experiments. We also thank Nostrum Biodiscovery for assistance with verification and for feedback on the practical relevance of this work.

References

[1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html

[2] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200. [Online]. Available: https://doi.org/10.1109/icse55347.2025.00157

[3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754–768. [Online]. Available: https://doi.org/10.1109/sp46214.2022.9833571

[4] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2785–2799. [Online]. Available: https://doi.org/10.1145/3576915.3623157

[5] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 8, pp. 218:1–218:34, 2025. [Online]. Available: https://doi.org/10.1145/3716848

[6] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, "Is vibe coding safe? benchmarking vulnerability of agent-generated code in real-world tasks," in *Forty-third International Conference on Machine Learning*, 2026, author-provided conference citation; benchmark release v1.0. [Online]. Available: https://openreview.net/forum?id=qG8g00zRZa

[7] LeiLiLab, "SusVibes: Benchmark and evaluation pipeline," Software repository, 2026, evaluator revision used in this study; accessed October 19, 2026. [Online]. Available: https://github.com/LeiLiLab/susvibes/tree/7e1b4b05240e56dc2b4e8253b2cc5c9481d016f3

[8] GitHub, "CodeQL CLI," GitHub documentation, accessed October 19, 2026. [Online]. Available: https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-cli

[9] Semgrep, "Semgrep code overview," Semgrep documentation, accessed October 19, 2026; product capabilities vary by engine and configuration. [Online]. Available: https://docs.semgrep.dev/semgrep-code/overview

[10] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 672–681. [Online]. Available: https://doi.org/10.1109/icse.2013.6606613

[11] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," *Communications of the ACM*, vol. 61, no. 4, pp. 58–66, 2018. [Online]. Available: https://doi.org/10.1145/3188720

[12] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. O'Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, "Moving Fast with Software Verification," in *NASA Formal Methods*, 2015, pp. 3–11. [Online]. Available: https://doi.org/10.1007/978-3-319-17524-9_1

[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=9LdJDU7E91

[14] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, "LLMDFA: Analyzing Dataflow in Code with Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 131 545–131 574. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/ed9dcde1eb9c597f68c1d375bbecf3fc-Abstract-Conference.html

[15] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 21 083–21 100. [Online]. Available: https://proceedings.mlr.press/v267/guo25n.html

[16] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "InferFix: End-to-End Program Repair with LLMs," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1646–1656. [Online]. Available: https://doi.org/10.1145/3611643.3613892

[17] A. Desai, M. Shafiuzzaman, W. Guo, and T. Bultan, "Program analysis guided LLM agent for proof-of-concept generation," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2604.07624

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[19] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014, pp. 313–324. [Online]. Available: https://doi.org/10.1145/2642937.2642982

[20] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html

[21] T. Lanham, A. Chen, A. Radhakrishnan, B. Steiner, C. Denison, D. Hernandez, D. Li, E. Durmus, E. Hubinger, J. Kernion, K. Lukošiūtė, K. Nguyen, N. Cheng, N. Joseph, N. Schiefer, O. Rausch, R. Larson, S. McCandlish, S. Kundu, S. Kadavath, S. Yang, T. Henighan, T. Maxwell, T. Telleen-Lawton, T. Hume, Z. Hatfield-Dodds, J. Kaplan, J. Brauner, S. R. Bowman, and E. Perez, "Measuring Faithfulness in Chain-of-Thought Reasoning," 2023, preprint. [Online]. Available: https://arxiv.org/abs/2307.13702

[22] J. Zhou, M. Pacheco, J. Chen, X. Hu, X. Xia, D. Lo, and A. E. Hassan, "CoLeFunDa: Explainable silent vulnerability fix identification," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2565–2577. [Online]. Available: https://xing-hu.github.io/assets/papers/icse23colefunda.pdf

[23] Ivan, "Undoing the fix: Evaluating AI control against security regression attacks," Apart Research AI Control Hackathon report, Mar. 2026, author credited as Ivan in the supplied report; exploratory study. [Online]. Available: https://github.com/strivn/undoing-the-fix/tree/6511beccd75b63c93628ec8dc062ebb07a0bcc5d

[24] J. L. Watson, D. Juergens, N. R. Bennett, B. L. Trippe, J. Yim, H. E. Eisenach, W. Ahern, A. J. Borst, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, N. Hanikel, S. J. Pellock, A. Courbet, W. Sheffler, J. Wang, P. Venkatesh, I. Sappington, S. V. Torres, A. Lauko, V. De Bortoli, E. Mathieu, S. Ovchinnikov, R. Barzilay, T. S. Jaakkola, F. DiMaio, M. Baek, and D. Baker, "De novo design of protein structure and function with RFdiffusion," *Nature*, vol. 620, pp. 1089–1100, 2023. [Online]. Available: https://doi.org/10.1038/s41586-023-06415-8

[25] J. Dauparas, I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, A. Courbet, R. J. de Haas, N. Bethel, P. J. Y. Leung, T. F. Huddy, S. Pellock, D. Tischer, F. Chan, B. Koepnick, H. Nguyen, A. Kang, B. Sankaran, A. K. Bera, N. P. King, and D. Baker, "Robust deep learning–based protein sequence design using ProteinMPNN," *Science*, vol. 378, no. 6615, pp. 49–56, 2022. [Online]. Available: https://doi.org/10.1126/science.add2187

[26] M. Giulini, V. Reys, J. M. C. Teixeira, B. Jiménez-García, R. V. Honorato, A. Kravchenko, X. Xu, R. Versini, A. Engel, S. Verhoeven, and A. M. J. J. Bonvin, "HADDOCK3: A Modular and Versatile Platform for Integrative Modeling of Biomolecular Complexes," *Journal of Chemical Information and Modeling*, vol. 65, no. 13, pp. 7315–7324, 2025. [Online]. Available: https://doi.org/10.1021/acs.jcim.5c00969

[27] C. Mirabello and B. Wallner, "DockQ v2: improved automatic quality measure for protein multimers, nucleic acids, and small molecules," *Bioinformatics*, vol. 40, no. 10, p. btae586, 2024. [Online]. Available: https://doi.org/10.1093/bioinformatics/btae586

[28] J. Eberhardt, D. Santos-Martins, A. F. Tillack, and S. Forli, "AutoDock Vina 1.2.0: New Docking Methods, Expanded Force Field, and Python Bindings," *Journal of Chemical Information and Modeling*, vol. 61, no. 8, pp. 3891–3898, 2021. [Online]. Available: https://doi.org/10.1021/acs.jcim.1c00203

[29] M. van Kempen, S. S. Kim, C. Tumescheit, M. Mirdita, J. Lee, C. L. M. Gilchrist, J. Söding, and M. Steinegger, "Fast and accurate protein structure search with Foldseek," *Nature Biotechnology*, vol. 42, pp. 243–246, 2024. [Online]. Available: https://doi.org/10.1038/s41587-023-01773-0

[30] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," National Institute of Standards and Technology, Tech. Rep. SP 800-218, 2022. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[31] GumTreeDiff, "GumTree, release 3.0.0," Software release, accessed October 19, 2026. [Online]. Available: https://github.com/GumTreeDiff/gumtree/releases/tag/v3.0.0

[32] Python Software Foundation, "ast—Abstract Syntax Trees," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/ast.html

[33] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*.

Chapman and Hall/CRC, 1994. [Online]. Available: https://doi.org/10.1201/9780429246593

[34] PyTorch Contributors, "Serialization API and Safe-Globals Context Manager," PyTorch v2.5.1 source and API documentation, 2024, pinned version used by the component study; accessed October 20, 2026. [Online]. Available: https://github.com/pytorch/pytorch/blob/v2.5.1/torch/serialization.py

[35] Python Software Foundation, "pickle—Python Object Serialization," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/pickle.html

[36] ——, "tokenize—Tokenizer for Python Source," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/tokenize.html

[37] M. Monperrus, "Automatic Software Repair: A Bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 17:1–17:24, 2018. [Online]. Available: https://doi.org/10.1145/3105906

[38] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543. [Online]. Available: https://doi.org/10.1145/2786805.2786825

[39] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[40] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39. [Online]. Available: https://doi.org/10.1145/3475960.3475985

[41] Q.-C. Bui, R. Scandariato, and N. E. D. Ferreyra, "Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 464–468. [Online]. Available: https://doi.org/10.1145/3524842.3528482

[42] P. Yu, Y. Wu, X. Peng, J. Peng, J. Zhang, P. Xie, and W. Zhao, "ViolationTracker: Building precise histories for static analysis violations," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023. [Online]. Available: https://conf.researchr.org/details/icse-2023/icse-2023-technical-track/12/ViolationTracker-Building-Precise-Histories-for-Static-Analysis-Violations

[43] C. Yang, Z. Zhao, Z. Xie, H. Li, and L. Zhang, "KNighter: Transforming static analysis with LLM-synthesized checkers," in *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025. [Online]. Available: https://arxiv.org/abs/2503.09002

[44] A. Koyuncu, K. Liu, T. F. Bissyandé, D. Kim, J. Klein, M. Monperrus, and Y. Le Traon, "FixMiner: Mining relevant fix patterns for automated program repair," *Empirical Software Engineering*, vol. 25, no. 3, pp. 1980–2024, 2020. [Online]. Available: https://doi.org/10.1007/s10664-019-09780-z

[45] C. S. Xia and L. Zhang, "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 819–831. [Online]. Available: https://doi.org/10.1145/3650212.3680323

[46] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[47] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[48] motornomad, "POSTURA: Agentic attack surface posture intelligence," Software repository, repository owner handle; accessed October 19, 2026. [Online]. Available: https://github.com/motornomad/postura

[49] P. Rajput and S. Khan, "Securing Agent-Generated Code Through Development History," Zenodo, 2026, software and derived study evidence. [Online]. Available: https://doi.org/10.5281/zenodo.23530070